Del 1:

Introduksjon til Databaser

Christian Marker
Lead Software Developer
Digital Telco

Hva skal vi lære i dag?

- 📋 **Tabeller** – Grunnsteinen i en database
- 🔤 **Kolonnetyper** – Hvilke typer data kan vi lagre?
- 🔗 **Relasjoner** – Hvordan henger data sammen?
- 🔑 **Primær- og fremmednøkler** – Kobling mellom tabeller
- 🔍 **SELECT** – Hente og filtrere data
- 🔗 **JOIN** – Hente data fra flere tabeller samtidig
- ✏️ **Oppgaver** – Opprette tabeller og skrive spørringer
- *— LUNSJ —*
- Hvordan koble opp mot en database i .NET?

Tabeller

Grunnsteinen i en database

Hva er en tabell?

En tabell er den grunnleggende strukturen for å lagre data i en database.

En tabell består av:

- **Rader** – én enhet av data
- **Kolonner** – en egenskap ved dataene
- **Celler** – verdien der en rad og kolonne møtes

💡 Tenk på en tabell som et **Excel-ark** – men med strenge regler for hva slags data som kan legges inn!

Eksempel: Tabell med studenter

```sql
-- Opprette tabellen
CREATE TABLE studenter (
    id          INTEGER,
    fornavn     VARCHAR(50),
    etternavn   VARCHAR(50),
    epost       VARCHAR(100),
    fodselsar   INTEGER
);
```

id	fornavn	etternavn	epost	fodselsar
1	Kari	Nordmann	kari@skole.no	2005
2	Ola	Hansen	ola@skole.no	2004
3	Ingrid	Berg	ingrid@skole.no	2005

Kolonnetyper

Hvilke typer data kan vi lagre?

Hva er kolonnetyper?

Hver kolonne i en tabell må ha en **datatype** som bestemmer hva slags verdier som kan lagres der.

Dette er viktig fordi det:

- Sikrer at riktig type data lagres
- Gjør databasen mer effektiv
- Forhindrer feil (f.eks. tekst der det skal være tall)

Vanlige kolonnetyper (1/2)

Tekst

Type	Beskrivelse
`CHAR(n)`	Fast lengde (n) tegn
`VARCHAR(n)`	Variabel lengde tekst (maks n tegn)
`TEXT`	Ubegrenset tekst

Tall

Type	Beskrivelse
`INTEGER`	Heltall (…-1, 0, 1, 2…)
`DECIMAL(p,s)`	Desimaltall med presisjon
`FLOAT`	Flyttall, når presisjon ikke er kritisk

Vanlige kolonnetyper (2/2)

Dato og tid

Type	Beskrivelse
`DATE`	Dato (YYYY-MM-DD)
`TIME`	Klokkeslett
`DATETIME`	Dato og tid kombinert

Andre

Type	Beskrivelse
`BOOLEAN`	Sann/Usann
`NULL`	Ingen verdi / ukjent

Kolonnetyper i praksis

```sql
CREATE TABLE produkter (
    id            INTEGER,          -- heltall: 1, 2, 3 ...
    navn          VARCHAR(100),     -- tekst: maks 100 tegn
    beskrivelse   TEXT,             -- lang tekst uten grense
    pris          DECIMAL(10, 2),   -- f.eks. 199.90
    på_lager      BOOLEAN,          -- TRUE eller FALSE
    opprettet     DATE              -- f.eks. 2024-01-15
);
```

💡 **NB:** I systemer med begrenset lagringskapasitet er det viktig å velge rett type på teskt-verdiene. Men i det daglige så vil det være nok med `TEXT`.

Relasjoner

Hvordan henger data sammen?

Hva er relasjoner? (1/2)

En relasjonsdatabase lar oss koble tabeller sammen slik at vi unngår å gjenta de samme dataene mange ganger. Dette kalles **normalisering**.

Uten relasjoner (dårlig):

ordre_id	kunde_navn	kunde_epost	vare
1	Kari N.	kari@skole.no	Bok
2	Kari N.	kari@skole.no	Penn
3	Ola H.	ola@skole.no	Linjal

Data om Kari gjentas på hver rad - dette er sløsing og kan gi inkonsistens.

Hva er relasjoner? (2/2)

Tabell: `kunder`

id	navn	epost
1	Kari N.	kari@skole.no
2	Ola H.	ola@skole.no

Tabell: `produkter`

id	navn	pris
1	Bok	99
2	Penn	15
3	Linjal	25

Tabell: `ordrer`

id	kunde_id	produkt_id
1	1	1
2	1	2
3	2	3

Primær- og fremmednøkler

Kobling mellom tabeller

Primærnøkkel (Primary Key)

En **primærnøkkel** er en kolonne (eller kombinasjon av kolonner) som **unikt identifiserer** hver rad i en tabell.

```
CREATE TABLE studenter (
    id        INTEGER     PRIMARY KEY AUTO_INCREMENT,  -- primærnøkkel!
    fornavn   VARCHAR(50) NOT NULL,
    etternavn VARCHAR(50) NOT NULL,
    epost     VARCHAR(100)
);
```

Regler for primærnøkler:

- Må være **unik** – ingen to rader kan ha samme verdi
- Kan **ikke** være `NULL` (tom/ukjent)
- Hver tabell bør ha minst **én** primærnøkkel
- Ofte brukes et automatisk incrementerende tall: `SERIAL` / `AUTO_INCREMENT`

Fremmednøkkel (Foreign Key)

En **fremmednøkkel** er en kolonne som refererer til primærnøkkelen i en annen tabell – og skaper dermed en kobling mellom tabellene.

```sql
CREATE TABLE klasser (
    id    INTEGER     PRIMARY KEY,
    navn  VARCHAR(50) NOT NULL
);

CREATE TABLE studenter (
    id        INTEGER     PRIMARY KEY,
    fornavn   VARCHAR(50) NOT NULL,
    klasse_id INTEGER     REFERENCES klasser(id)  -- fremmednøkkel!
);
```

🔗 `klasse_id` i `studenter`-tabellen peker på `id` i `klasser`-tabellen. Databasen sørger for at du ikke kan legge inn en `klasse_id` som ikke finnes!

Primær- og fremmednøkler visualisert

Tabell: `klasser`

id 🔑	navn
1	1A
2	1B
3	2A

Tabell: `studenter`

id 🔑	fornavn	klasse_id 🔗
1	Kari	1
2	Ola	1
3	Ingrid	2

`studenter.klasse_id` → `klasser.id`

Kari og Ola går i klasse **1A**, Ingrid går i klasse **1B**

Oppsummering

📋 **Tabeller** Grunnstrukturen i en database. Rader = poster, kolonner = egenskaper.

🔤 **Kolonnetyper** Hver kolonne har en type: `INTEGER`, `VARCHAR`, `DATE`, `BOOLEAN`, osv.

🔗 **Relasjoner** Unngå å gjenta data – koble tabeller i stedet for å duplisere informasjon.

🔑 **Primærnøkkel** Unik identifikator for hver rad i en tabell. Kan ikke være `NULL`.

🔗 **Fremmednøkkel** Refererer til en primærnøkkel i en annen tabell og skaper en kobling.

Oppgave 1 *(10 minutter)*

I deres prosjekt har dere følgende tabeller `Post`, `Comment` og `User` Skriv SQL-spørringer som oppretter disse tabellene.

Post	Comment	User
Id	Id	Id
Content	Content	Name
CreatedAt	CreatedAt	
Author	Author	
Title	PostId	
Comments		

Løsningsforslag: Oppgave 1

```sql
CREATE TABLE User (
    Id        INTEGER       PRIMARY KEY,
    Name      TEXT          NOT NULL
);

CREATE TABLE Post (
    Id        INTEGER       PRIMARY KEY,
    Content   TEXT          NOT NULL,
    CreatedAt DATE          NOT NULL,
    Author    INTEGER       NOT NULL REFERENCES User(Id),
    Title     TEXT          NOT NULL
);

CREATE TABLE Comment (
    Id        INTEGER       PRIMARY KEY,
    Content   TEXT          NOT NULL,
    CreatedAt DATE          NOT NULL,
    Author    INTEGER       NOT NULL REFERENCES User(Id),
    PostId    INTEGER       NOT NULL REFERENCES Post(Id)
);
```

SELECT

Hente og filtrere data

SELECT – grunnleggende syntaks

`SELECT` brukes til å hente data fra en tabell.
For å hente alle kolonner bruker man `*`

```
SELECT *
FROM studenter;
```

Resultat:

id	fornavn	etternavn	epost	fodselsar
1	Kari	Nordmann	kari@skole.no	2005
2	Ola	Hansen	ola@skole.no	2004
3	Ingrid	Berg	ingrid@skole.no	2005

SELECT – filtrere med WHERE

`WHERE` lar deg filtrere hvilke rader som returneres.

```
SELECT fornavn, etternavn, fodselsar
FROM studenter
WHERE fodselsar = 2005;
```

Resultat:

fornavn	etternavn	fodselsar
Kari	Nordmann	2005
Ingrid	Berg	2005

💡 Du kan kombinere betingelser med `AND` og `OR`, og bruke operatorer som `=`, `>`, `<`, `!=`, `LIKE`.

SELECT – sortere og begrense

`ORDER BY` sorterer resultatet, `LIMIT` begrenser antall rader.

```
SELECT fornavn, etternavn, fodselsar
FROM studenter
ORDER BY fodselsar ASC
LIMIT 2;
```

Resultat (eldste først, maks 2 rader):

fornavn	etternavn	fodselsar
Ola	Hansen	2004
Ingrid	Berg	2005

JOIN

Hente data fra flere tabeller samtidig

Hva er JOIN?

`JOIN` lar oss hente data fra **to eller flere tabeller** i én spørring, ved å koble dem sammen på en felles nøkkel.

```
SELECT kolonne1, kolonne2, ...
FROM tabell1
JOIN tabell2 ON tabell1.nokkel = tabell2.nokkel;
```

`JOIN` er en forkortelse for `INNER JOIN` – de betyr det samme.

🔍 `JOIN` returnerer **kun rader der det finnes en match i begge tabeller.** Rader uten kobling blir utelatt.

JOIN – Eksempel

Vi ønsker å vise studentenes navn **og** klassen de tilhører:

```
SELECT
    studenter.fornavn,
    studenter.etternavn,
    klasser.navn AS klasse
FROM studenter
JOIN klasser ON studenter.klasse_id = klasser.id;
```

Resultat:

fornavn	etternavn	klasse
Kari	Nordmann	1A
Ola	Hansen	1A
Ingrid	Berg	1B

JOIN – Visualisert

Tabell: `studenter`

id	fornavn	klasse_id
1	Kari	1
2	Ola	1
3	Ingrid	2
4	Per	NULL

Tabell: `klasser`

id	navn
1	1A
2	1B
3	2A

Resultat (INNER JOIN)

fornavn	klasse
Kari	1A
Ola	1A
Ingrid	1B

Per (ingen klasse) og 2A (ingen studenter) er ikke med i resultatet

JOIN med tre tabeller

Vi kan også koble sammen flere tabeller i én spørring:

```
SELECT
    studenter.fornavn,
    klasser.navn      AS klasse,
    laerere.navn      AS laerer
FROM studenter
JOIN klasser ON studenter.klasse_id = klasser.id
JOIN laerere ON klasser.laerer_id   = laerere.id;
```

💡 Du kan lenke så mange `JOIN` som du trenger – bare sørg for at du alltid kobler via nøklene!

Typer JOIN

Type	Beskrivelse	Inkluderer rader uten match
`INNER JOIN`	Kun rader med match i **begge** tabeller	Nei
`LEFT JOIN`	Alle rader fra **venstre** tabell	Ja, fra venstre (høyre får `NULL`)
`RIGHT JOIN`	Alle rader fra **høyre** tabell	Ja, fra høyre (venstre får `NULL`)
`FULL JOIN`	Alle rader fra **begge** tabeller	Ja, begge sider kan få `NULL`

💡 I praksis brukes `INNER JOIN` og `LEFT JOIN` mest.

Oppgave 2 *(10 minutter)*

Bruk tabellene du opprettet i forrige oppgave (`Post`, `Comment`, `User`).

Skriv en SQL-spørring som henter ut **alle poster skrevet av en bestemt bruker**, inkludert brukerens navn.

💡 **Tips:**

- Bruk `SELECT` for å velge hvilke kolonner du vil ha med
- Bruk `JOIN` for å koble `Post` og `User`
- Bruk `WHERE` for å filtrere på en bestemt bruker

Løsningsforslag: Oppgave 2

```sql
SELECT *
 -- Eller spesifikt:
SELECT
    Post.Title,
    Post.Content,
    Post.CreatedAt,
    User.Name
FROM Post
JOIN User ON Post.Author = User.Id
WHERE User.Name = 'Kari Nordmann';
```

Flere oppgaver i `databaser-1.md` frem til lunsj

Del 2:

Database i .NET API?

Koble til en database i .NET

Entity Framework Core er et ORM-bibliotek som lar deg jobbe med databasen via C#-klasser i stedet for rå SQL.

1. Installer NuGet-pakker

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

✅ Disse er allerede lagt til i prosjektet dere jobber med, så dere trenger ikke å gjøre dette manuelt.

Bare kjør `dotnet restore` for å hente inn pakkene.

Databaser i .NET (1/4)

2. **ApplicationDbContext** (`Data/AppDbContext.cs`) Her definerer du tabeller og relasjoner

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
    { }

    public DbSet<Post> Posts { get; set; } = null!;
    public DbSet<Comment> Comments { get; set; } = null!;
    public DbSet<User> Users { get; set; } = null!;

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // Configure Post entity
        modelBuilder.Entity<Post>(entity => { .. });
        // Configure Comment entity
        modelBuilder.Entity<Comment>(entity => { .. });
        // Configure User entity
        modelBuilder.Entity<User>(entity => { .. });
    }
}
```

Databaser i .NET (2/4)

```
// Configure Post entity
modelBuilder.Entity<Post>(entity =>
{
    // Define primary key
    entity.HasKey(e => e.Id);

    // Configure the relationship: Post has many Comments
    entity.HasMany(p => p.Comments)
        .WithOne()
        .HasForeignKey(c => c.PostId)
        .OnDelete(DeleteBehavior.Cascade);

    // Configure the relationship: Post has one Author (User)
    entity.HasOne(p => p.Author)
        .WithMany()
        .HasForeignKey(p => p.AuthorId);
});
```

Her konfigurerer vi `Post`-entiteten.

Vi trenger bare å definere relasjoner eksplisitt.

EF Core oppdager enkle kolonner som `Title` og `Content` automatisk basert på C#-klassen.

Oppgave 3 *(5-10 minutter)*

Fullfør `OnModelCreating`-metoden i `ApplicationDbContext`.

Definer databasetabellene for `User` og `Comment` på samme måte som `Post`.

Databaser i .NET (4/4) - Løsningsforslag

```
modelBuilder.Entity<User>(entity =>
{
    entity.HasKey(u => u.Id);
});

// Configure Comment entity
modelBuilder.Entity<Comment>(entity =>
{
    entity.HasKey(e => e.Id);

    entity.HasOne(c => c.Author)
        .WithMany()                         // A User can have many Comments
        .HasForeignKey(c => c.AuthorId);    // Foreign key to User
});
```

Entity Framework: Konfigurasjon

3. Legg til connection string i `appsettings.json`

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=MiniRedditDb;Username=miniReddit;Password=localPassword123"
  }
}
```

4. Registrer DbContext i `Program.cs`

```
// Legg til DbContext med riktig connection string
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    // Hent connection string fra appsettings.json
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));

// Bytt ut AddSingleton med AddScoped for å få en ny DbContext per request.
builder.Services.AddScoped<IDatabaseService, DatabaseService>();
```

✅ Dette er allerede gjort for dere i prosjektet, så dere trenger ikke å gjøre dette manuelt.

Entity Framework: Modeller og migrasjoner

5. Liten endring i `BaseEntity`:

```
public class BaseEntity
{
    public int Id { get; set; }
    public string? Content { get; set; }
    public DateTime CreatedAt { get; set; }
    public string AuthorId { get; set; } = string.Empty;
    public required User Author { get; set; }
}
```

6. Opprett og kjør migrasjoner

```
dotnet ef migrations add InitialCreate --project MiniReddit
dotnet ef database update --project MiniReddit
```

💡 EF Core genererer SQL automatisk fra C#-klassene dine. Du slipper å skrive `CREATE TABLE` for hånd!

Entity Framework: Spørringer (1/3)

Når databasen er satt opp kan du hente data med EF Core i stedet for SQL:

```
// Hent alle poster av en bestemt bruker
var poster = await db.Posts
    .Include(p => p.Author)
    .Where(p => p.Author.Name == "Kari Nordmann")
    .ToListAsync();
```

SQL	Entity Framework
`SELECT * FROM Post`	`db.Posts`
`JOIN`	`.Include(p => p.Author)`
`WHERE`	`.Where(p => ...)`

`.ToListAsync()` henter resultatet som en liste i C#.

Entity Framework: Spørringer (2/3)

SQL	Entity Framework
`SELECT col1, col2`	`.Select(p => new { p.Col1, p.Col2 })`
`ORDER BY col ASC`	`.OrderBy(p => p.Col)`
`ORDER BY col DESC`	`.OrderByDescending(p => p.Col)`
`LIMIT 10`	`.Take(10)`
`OFFSET 20`	`.Skip(20)`

Entity Framework: Spørringer (3/3)

Hittil har vi kun fokusert på å hente data. Men for å legge til, oppdatere eller slette data så har vi følgende metoder:

SQL	Entity Framework
`INSERT INTO`	`db.Add(entity)`
`UPDATE ... WHERE`	hent med `.FirstAsync(p => ...)`, endre felt, kall `db.Update(entity)`
`DELETE ... WHERE`	hent med `.FirstAsync(p => ...)`, kall `db.Remove(entity)`

Opprette:

```
var post = new Post { Title = "Hei" };
db.Add(post);
await db.SaveChangesAsync();
```

Oppdater:

```
var post = await db.Posts
    .FirstAsync(p => p.Id == 1);
post.Title = "Ny tittel";
db.Update(post);
await db.SaveChangesAsync();
```

Slett:

```
var post = await db.Posts
    .FirstAsync(p => p.Id == 1);
db.Remove(post);
await db.SaveChangesAsync();
```

Flere oppgaver i `databaser-2.md` ut dagen